# AI Game Recommendation Agent

## Problem

The video game industry is massive and thus, there are many, many video games to choose from. From different genres, difficulty, length, varying costs; it can be hard to find the games that you **want** to play. This application solves this problem, by creating an AI Agent that learns your preferences and personalises recommendations through a users behaviour AND through their direct action.

## Target Users:

- Users who play video games that want a better way to find games.
- Users who do not currently play games, but have an interest in trying out games

## What can the Agent Do?

- Creates a Personalized Catalog of games that the user would be interested in
- Frequently updates the users preferences based of habits and direct data
- Able to recommend games through regular prompts (outside of the personalised catalog)
- Connect to game libraries like Steam to give a more personalized experience
- Provide relevant gaming news the user
- Recommend games based off friends list or other users
- Estimate how long a game would take based off the users previous experiences with games.

**Why AI:** It is hard for regular programs to understand nuance. When an individual has specific requests that are outside of the bounds of non-AI programs, the user is left in a rut. Using AI solves this problem as it can understand nuanced requests as it can understand natural language and even “slang” terms.

**Primary LLM Model:** ChatGPT

- Used for understanding the prompts from users
- Interpreting user preferences
- Identify connections between different games
- Generating explanations on why games were recommended
- Refine user preferences based off user behaviour

# Feature Specification

**F01 - Game recommendation catalog**

- **Description:** Creates a personalized catalog based off the user's preferences
- **User Interaction:** User completes modules to refine their preferences. Regular interaction with other features (feedback, prompting, etc) also refine preferences that refine what is shown in the catalog
- **Output:** List of recommended games with key details and explanations for the recommendation itself
- **AI Involvement:** Hybrid
- **Workflow:** Collect Information -> Refine Profile -> Retrieve Games -> Display Results
- **Errors:**
    - Poor Recommendations -> request user for more information on their preferences
    - LLM Service Down -> Display error on home screen

**F02 - User Profile/Preferences**

- **Description:** Showcases the User's preferences that were generated by the AI Agent. Allows direct adjustment to these preferences by the user.
- **User Interaction:** User can read through and adjust their preferences manually
- **Input:** Data from the AI Agent, Initial preferences, third-party connections, user-made changes
- **Output:** Updates the User's Profile
- **AI Involvement:** Hybrid
- **Workflow:** Collect information/Manually Input Information -> Refine Profile -> Recommendation services notified
- **Errors:** Conflicting preferences -> Ask the user to prioritize one of the other

**F03 - User Feedback**

- **Description:** Users can give feedback to games they have played; give ratings, reviews, and relevant data about the game.
- **User Interaction:** Buttons and forms that allow the user to review the games they've played
- **Input:** Forms/Buttons
- **Output:** Updated user preferences to refine game recommendations
- **AI Involvement:** Hybrid
- **Workflow:** Feedback collected -> refine preferences -> recommendation services are notified
- **Errors:** Conflicting feedback -> 2 very similar games with conflicting feedback, ask the user to refine why the feedback was given.

**F04 - Steam Integration**

- **Description:** Allows the user to connect their steam profile to their Steam account (Major Video Game Marketplace)
- **User Interaction:** Selecting a "Connect Steam" and inputting relevant info
- **Input:** Steam authentication and information
- **Output:** User profile information, user's current game catalog, achievements, etc.
- **AI Involvement:** Hybrid
- **Workflow:** User Inputs data -> retrieve steam data -> update user preferences -> notify recommendation services
- **Errors:** Steam Connection Fail -> display error, send feedback option and ability to retry

**F05 - Preference Refinement Modules**

- **Description:** Follow-up to initial modules, an optional more detailed module that gives more refinement to the users preferences
- **User Interaction:** User chooses between doing it or not, fills out relevant information on module
- **Input:** Accept or Decline, user responds to questions.
- **Output:** A more refined profile

- **AI Involvement:** AI
- **Workflow:** Agent identifies gaps in preferences -> chooses questions to ask the user -> prompts user -> uses data to update profile -> notifies recommendation services
- **Errors:** AI asks irrelevant question -> user can send feedback

**F06 - Curveball Roulette System**

- **Description:** Basically a random rolling system. Where the application provides a random game based on current gaming trends along with optional user input
- **User Interaction:** User presses a button
- **Input:** User provided input and current gaming trends
- **Output:** 2 game recommendations: One that is very popular and one that is more “niche”
- **AI Involvement:** Hybrid
- **Workflow:** Search Gaming Trends -> incorporate some user preferences -> select 2 games -> provide games to user -> give explanations
- **Errors:** Not enough data -> Ask the user if they want to provide more info / Broaden search

**F07 - Gaming News Panel**

- **Description:** Provides news on current gaming trends relevant to the user
- **User Interaction:** Prompting and interacting with the gaming news section
- **Input:** User preferences, current gaming trends and user prompting
- **Output:** Relevant News feed
- **AI Involvement:** Hybrid
- **Workflow:** Analyze Preferences/Prompts -> API Call -> Find relevant news -> organize news into panels -> provide news to user
- **Errors:** Steam API/Relevant News Outlet APIs **down** -> use stored news, show time stamp of when news was acquired

**F08 - Gaming Calendar**

- **Description:** A weekly/monthly gaming itinerary of games to play
- **User Interaction:** User inputs relevant information, when they play games, how long, “vibe” during the session etc.
- **Input:** User availability, recommended games, user inputted games, playtime of each game (NOTE: Works with playtime estimator)
- **Output:** Schedule for Games to play and when
- **AI Involvement:** Hybrid
- **Workflow:** Organize relevant data -> API Calls -> retrieve games that match criteria well -> organize games into calendar -> provide calendar to user
- **Errors:** User “free time” is limited -> suggest user to either play very short games, split games into sessions or to alter their current schedule

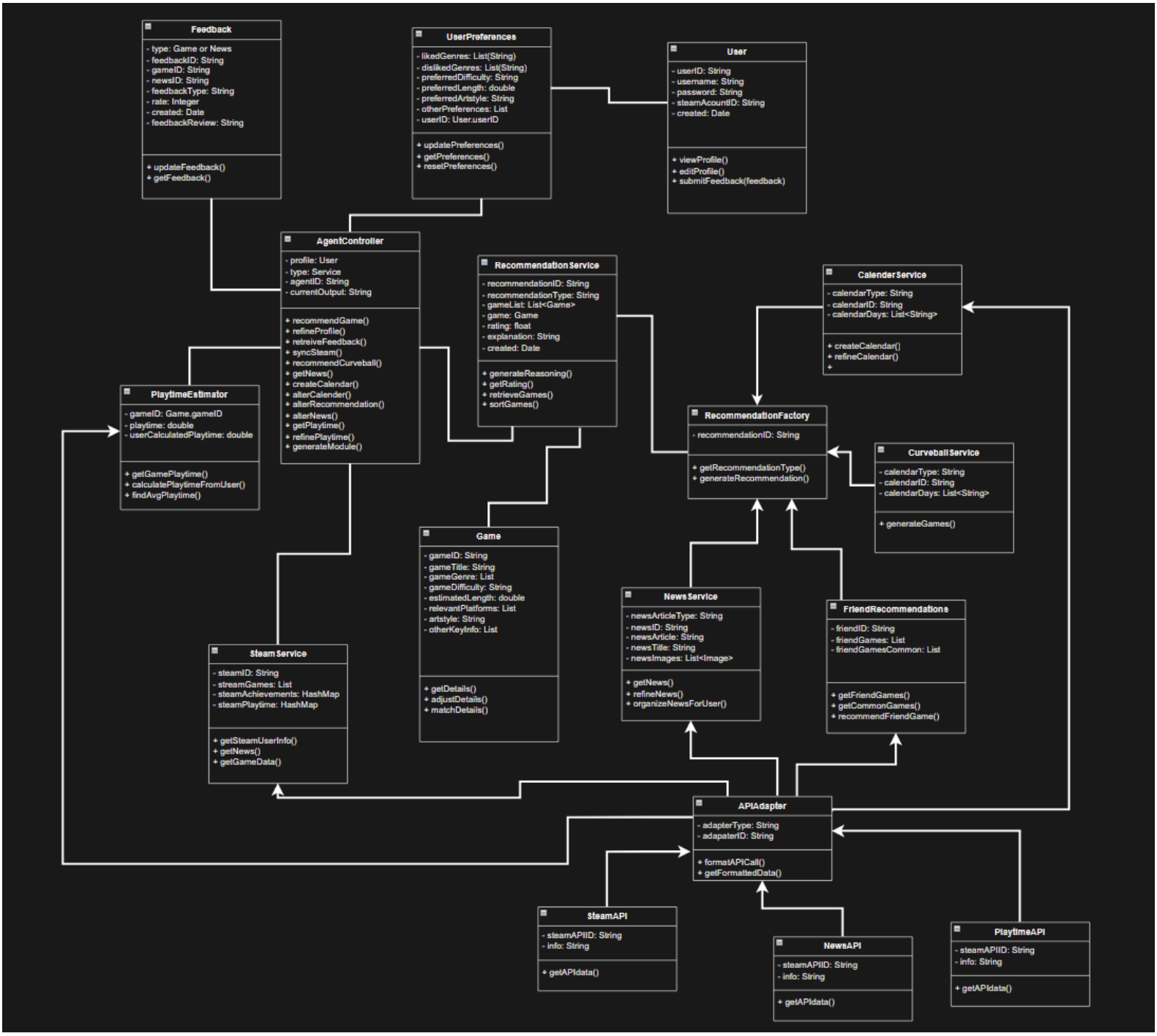
**F09 - Friend Game Recommendations**

- **Description:** Uses other users gaming information to provide games to the user
- **User Interaction:** Interacts with the friends section, gives the application specific users to get recommendations from
- **Input:** Other user information, current user preferences
- **Output:** List of relevant games that other users are playing (friends) that are relevant to the user
- **AI Involvement:** Hybrid
- **Workflow:** Retrieve data from relevant third party applications -> sort data -> compare data to user preferences -> provide user with relevant games
- **Errors:** Private Friend Account -> Asks the user to pick a different user to make recommendations from

**F10 - Playtime Estimator**

- **Description:** Provides the user with an estimated “time to beat” for them specifically, based off user data and third party information on a specific game.
- **User Interaction:** User must select a game and then choose the playtime estimator button
- **Input:** User data of previous games, third party info of game
- **Output:** **“**Time to beat” estimate on game
- **AI Involvement:** Hybrid
- **Workflow:** User selects game -> third party calls to get general playtime data -> search user data for previous games completion time -> compare and organize data -> output estimated time
- **Errors:** Not enough user data -> make the majority of the decision on general data from third party, let the user know this is the case.

**UML Class Diagram**



# Design Patterns

**Design Pattern 1 - Observer Pattern**

**(Classes used:** All Service Related Classes)

- Multiple objects in the application need to adjust when the user interacts with any other services
- AgentController acts a middle ground that interprets the new data, and then adjusts connected services/classes
- Without it, the User class itself would have to manually change preferences in each class

**Design Pattern 2 - Strategy Pattern**

**(Classes used:** RecommendationService, CurveballRecommendationService, FriendRecommendationService)

- Different types of recommendation situations then require different algorithms for each
- The RecommendationService can't change the its recommendation without changing its core principles/logical code
- Without it, the RecommendationService would be much larger and harder to be readable. Also is less efficient as it has to go through much more logic to get to the exact recommendation Algorithm

**Design Pattern 3 - Adapter Pattern**

**(Classes used:** NewsService, SteamService, PlaytimeService, APIAdapter)

- Third-party APIs have their own way of formatting data
- The APIAdpter allows the data to be formatted before it reaches other services
- Without it, we would have to with format it in every place we use it (repeat code) or use steam API syntax which might not be readable

**Design Pattern 4 - Facade Pattern**

**(Classes used:** AgentController, SteamService, NewsService, PlaytimeService)

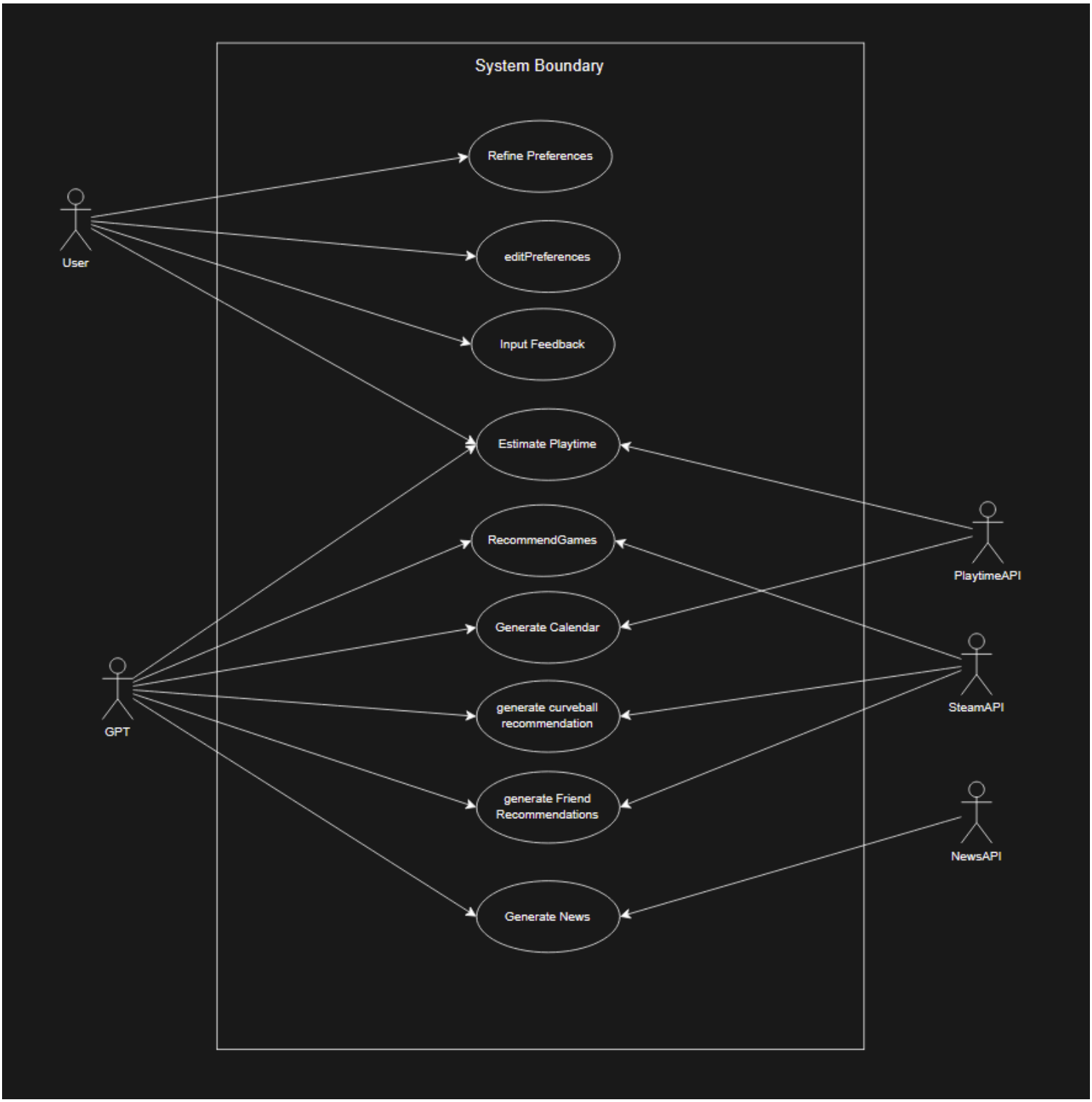
- The user indirectly needs to interact with a lot of services
- In this case, the AgentController is really the only service the user interacts with, as that then interacts with everything else
- If the user interacted with this services, we would have to add a lot of business logic to them, this way we centralize it to certain services.

**Design Pattern 5 - Factory Method**

**(Classes used:** RecommendationFactory - all RecommendationServices)

- We recommend different things for different situations
- RecommendationFactory creates each other recommendationService, to have a more organized view of each service.
- Also allows other recommendationServices to be made easier

# Use-Case Diagram



# Detailed Use-Case's

**UC1 - Generate Recommendations**

- **Actors:** User, ChatGPT, SteamAPI, PlaytimeService
- **Goal:** Generate game recommendations
- **Preconditions:** user has an account
- **Trigger:** user first interaction with service, continually in flux as user interacts with the platform
- **Main Success Scenario:**
    - User fills out initial module
    - Agent interprets the information to create UserPreferences
    - Agent then use RecommendationService to provide games that the user would like
    - Display games to user
    - User continually feeds the application information to refine UserPreferences
    - More refined game recommendations
- **Alternatives:** No game matches the criteria -> Agent broadens its search
- **Postcondition:** Games are displayed and stored in user history
- **Related: F01, F05, F06, F08, F09**

**UC2 - Refine Profile**

- **Actors:** User, ChatGPT
- **Goal:** Improves the accuracy of the recommendation service
- **Preconditions:** User has a preestablished profile and preferences
- **Trigger:** User selects a way to refine their preferences
- **Main Success Scenario:**
    - Agent finds gaps in preferences
    - Agent generates questions for the user to answer
    - User answers the questions involved
    - Agent interprets the answers and refines preferences
    - All other relevant services are notified
- **Postcondition:** module data is stored and used for future recommendations
- **Alternative:** User does not activate module -> profile is unchanged
- **Related: F02**

**UC3 - Submitting Feedback**

- **Actors:** User, SteamAPI, ChatGPT
- **Goal:** Improve recommendations
- **Preconditions:** User has played the game in question, user has an account
- **Trigger:** User interacts with a game and chooses to submit feedback
- **Main Success Scenario:**
    - User likes/dislikes a game they played
    - User submitted written feedback for the game as well
    - Agent interprets information
    - Agent refines preferences for user
- **Postcondition:** Feedback is stored for review
- **Alternative:** no feedback given -> user preferences remain unchanged, can be changed through other actions
- **Related: F03**

**UC4 - Steam Connection**

- **Actors:** User, GPT, SteamAPI
- **Goal:** setup a connection with user's steam account for more personalization
- **Preconditions:** user must have an account, user must have a steam account
- **Trigger:** User clicks on "Connect Steam"
- **Main Success Scenario:**
    - User clicked on connect steam
    - Inputs relevant info to their account
    - Application establishes connection with Steam and retrieves relevant data
    - Data is used to improve preferences
    - Data is used and updated regularly
- **Postcondition:** readily established connection with the steam profile
- **Alternative:** no steam account -> user can still refine their profile and input information manually
- **Related: F04**

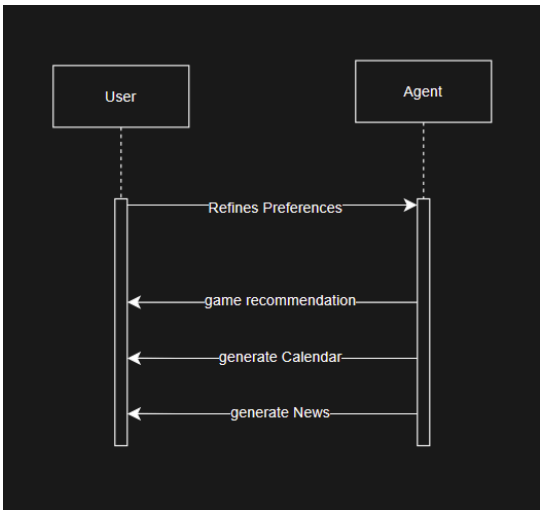
**UC5 - View Gaming News**

- **Actors:** Users, NewsService, GPT
- **Goal:** Generate news that is relevant to the user
- **Preconditions:** User has an account and generated preferences
- **Trigger:** User accesses the News tab
- **Main Success Scenario:**
    - User enter the News tab
    - User can pursue different news articles related to topics they are interested in
    - User can input feedback on News articles
    - Agent refines preferences for the user
- **Postcondition:** A full section of news articles
- **Alternative:** Not enough Preference data -> News tab is just a broad articles related to gaming
- **Related: F07**

**UC6 - Playtime Estimator**

- **Actors:** Users, PlaytimeService, GPT
- **Goal:** Gives an estimate on the playtime of a certain game
- **Preconditions:** User has an account and generated preferences
- **Trigger:** User accesses the playtime estimator tab, and enters a game to be estimated
- **Main Success Scenario:**
    - User enter the playtime tab
    - User inputs a game they want the playtime to be estimated
    - Agent searches through relevant third party and user data
    - Agent provides user with estimated playtime
- **Postcondition:** an estimated playtime for that game
- **Alternative:** Not enough Preference data -> gives a general idea of playtime, based off third party services
- **Related: F10**

## Sequence Diagram




## Feature-to-Design Traceability

| Feature | Description | Type | Related Use Case | Classes | Key Methods | Sequence Diagram | Design Patterns |
|---|---|---|---|---|---|---|---|
| **F01** | Generate video game recommendations | Hybrid | **UC1** | RecommendationServices, UserPreferences, AgentController | getPreferences(), generateRecommendations() | SD1 | Facade, Strategy |
| **F02** | The user's profile and preferences | Hybrid | **UC2** | User, UserPreferences | editProfile(), updatePreferences(), getPreferences() | SD1 | Observer |
| **F03** | Enables the user to input feedback for games | Hybrid | **UC3** | Feedback, AgentController, UserPreferences | updateFeedback(), getFeedback() | SD1 | Observer |
| **F04** | Links steam account to user account | Hybrid | **UC4** | SteamService, SteamAPI, User | getSteamUserInfo(), getNews(), getGameData() | SD1 | Adapter |
| **F05** | Refine user preferences through modules | AI | **UC1, UC2** | RecommendationServices, UserPreferences, AgentController | getPreferences(), generateModule() | SD1 | Observer |
| **F06** | Randomly roll a game recommendation | Hybrid | **UC1** | RecommendationServices, CurveballServices, AgentController | generateGames() | SD1 | Strategy, Factory |
| **F07** | Showcase relevant gaming news | Hybrid | **UC5** | NewsService, RecommendationService, AgentController | getNews(), oraganizeNewsforUser() | SD1 | Observer, Strategy |
| **F08** | Produce a calendar for what games to play and when | Hybrid | **UC1** | RecommendationService, CalendarService, AgentController | createCalendar() refineCalendar() | SD1 | Facade, Strategy |
| **F09** | Recommend games off other users (friends) | Hybrid | **UC1** | FriendRecommendations, RecommendationService, AgentController | getCommonGames() recommendFriendGame() | SD1 | Strategy |
| **F10** | Estimate the Playtime of a game the user chooses | Hybrid | **UC6** | PlaytimeService, AgentController PlaytimeAPI | getGamePlaytime() findAvgPlaytime() | SD1 | Strategy |

## Feature Implementation Explanations

**F01, F06, F07, F08, F09**

- **Classes:** AgentController, RecommendationService - all factory services, all API Classes, UserPreferences
- **Main methods:** getPreferences(), generateRecommendations()
- **GUI:** User inputs information into the GUI to then get interpreted by the AgentController. This information is then stored within user preferences and is used to generate games that the user may enjoy

**F02, F03, F05**

- **Classes:** UserPreferences, Feedback, AgentController
- **Main methods:** updatePrefences(), updateFeedback()
- **GUI:** User interacts with the GUI in order to gather more information for the AgentController to use. This information is then used to update preferences and generate better recommendations

**F04**

- **Classes:** SteamService, User
- **Main Method:** getSteamUserInfo(), getGameData()
- **GUI:** User links Steam profile to add more personalization to their profile, enabling more refined game recommendations and extra features

**F10**

- **Classes:** PlaytimeService, SteamService, UserPreferences, AgentController
- **Main Methods:** getGamePlaytime(), findAvgPlaytime()
- **GUI:** User inputs a game that they want to find the average game time for. Agentcontroller links all relevant information and displays the result